Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №9

“Паралельність, багатопроцесність та оптимізація”

Виконав:
студент 3 курсу
групи ФЕП-33с
Качмар Денис

Викладач:
Проф. Свелеба Сергій Андрійович

Львів - 2026

ХІД РОБОТИ

Тема: Паралельність, багатопроцесність та оптимізація

Мета: Ознайомитися з основними підходами до паралельного та конкурентного виконання програм у Python, дослідити вплив Global Interpreter Lock (GIL) на виконання CPU-bound та I/O-bound задач, навчитися використовувати threading, multiprocessing, ThreadPoolExecutor і ProcessPoolExecutor. Навчитися виявляти вузькі місця (bottlenecks) у Python-програмах за допомогою timeit, cProfile і tracemalloc, аналізувати час виконання та використання оперативної пам'яті, застосовувати бібліотеку NumPy для векторизації обчислень та caching для уникнення повторних дорогих операцій. Експериментально визначити найефективніший спосіб оптимізації для конкретної предметної області проєкту.

Варіант: 5 (Каталог автомобілів)

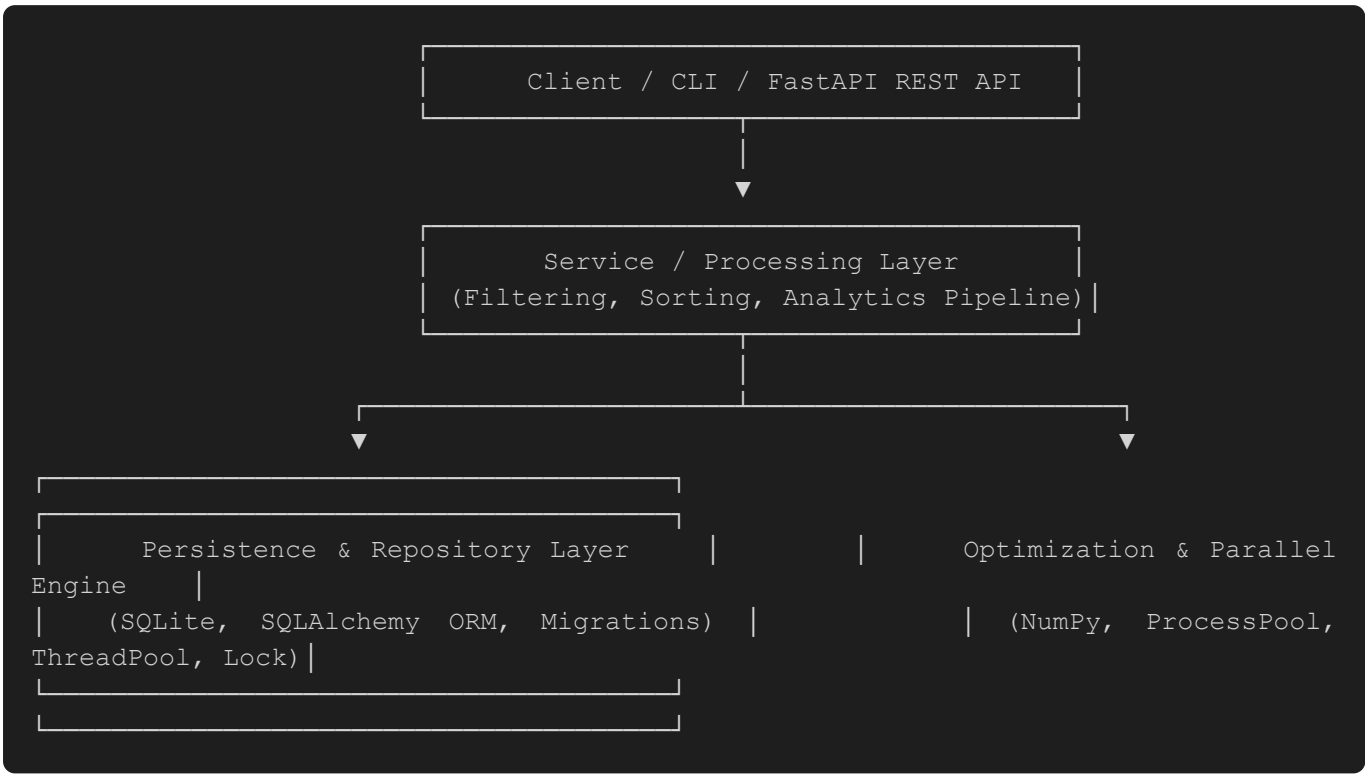
Постановка задачі:

1. Продовжити розвиток навчального проєкту «Каталог автомобілів» (Варіант 5) із попередніх лабораторних робіт №1–8. 2. Сформувати репрезентативні набори даних (10 000, 100 000, 200 000+ автомобілів) для профілювання та бенчмаркінгу. 3. Визначити та реалізувати базову CPU-bound операцію (розрахунок коефіцієнта енергоефективності/рейтингу автомобіля `calculate_efficiency_score`) та числові статистичні розрахунки каталогу (середнє, екстремуми, дисперсія, стандартне відхилення). 4. Провести інструментальне профілювання програми за допомогою `cProfile` та визначити вузькі місця (bottlenecks) на основі аналізу метрик `ncalls`, `tottime`, `cumtime`. 5. Дослідити споживання оперативної пам'яті (`current` та `peak memory`) через модуль `tracemalloc`. 6. Реалізувати та експериментально порівняти методи оптимізації: - Багатопотокова обробка (`threading`, `concurrent.futures.ThreadPoolExecutor`) та дослідження впливу GIL; - Забезпечення синхронізації (`threading.Lock`) для захисту спільного змінюваного стану від станів гонитви (`race conditions`); - Багатопроцесна паралельна обробка (`multiprocessing`, `concurrent.futures.ProcessPoolExecutor`) на базі багатоядерної архітектури; - Векторизована числова обробка за допомогою бібліотеки `NumPy`; - Мемоізація та кешування результатів за допомогою `@functools.lru_cache` із механізмом інвалідації кешу при зміні даних. 7. Провести серію вимірювань (щонайменше 3 розміри dataset та 4 конфігурації workers: 1, 2, 4, 8), розрахувати коефіцієнти прискорення (`speedup`) та перевірити математичну еквівалентність оптимізованих результатів базовій версії. 8. Оформити зведений академічний звіт із таблицями вимірювань, термінальними скриншотами та відповідями на всі контрольні запитання.

---

1. Короткий опис існуючого проєкту та архітектури

У межах лабораторних робіт № 1–8 було створено модульну систему обліку та аналітики транспортних засобів «Каталог автомобілів» (Варіант 5). Архітектура системи побудована відповідно до принципів Clean Architecture та Domain-Driven Design (DDD):



Сутність автомобіля (`Car`) характеризується атрибутами: унікальний `id` , марка (`brand`), модель (`model`), календарний рік випуску (`year`), ринкова вартість у доларах (`price`), експлуатаційний пробіг у кілометрах (`mileage`) та опціональний 17-значний VIN-номер.

У Лабораторній роботі № 9 система доповнена спеціалізованими модулями обчислювальної оптимізації: - `src/car_catalog/analytics.py` — векторні та скалярні статистичні розрахунки, математичний скоринг, генератори синтетичних даних та кешування; - `src/car_catalog/parallel.py` — паралельні обчислення через `ThreadPoolExecutor` та `ProcessPoolExecutor` , синхронізовані примітиви (`ThreadSafeCounter` на базі `Lock`); - `src/car_catalog/profiling.py` — утиліти профілювання `cProfile` , відстеження пам'яті `tracemalloc` , багаторазові бенчмарки та розрахунок `speedup` ; - `benchmarks/` — набір ізольованих модулів навантажувального тестування з автоматичним експортом у CSV.

---

2. Опис обраного Bottleneck

При масовому аналізі характеристик великого автопарку (від десятків тисяч до мільйонів авто) виникає потреба в складних аналітичних розрахунках: 1. **CPU-bound скоринг транспортного засобу (`calculate_efficiency_score`)**: алгоритм комплексної оцінки енергетичної та експлуатаційної ефективності автомобіля, що містить інтенсивні трансцендентні обчислення: $$\text{score} = \sum_{i=1}^{100} \left(\sin^2(v) + \cos^2(v) + \sqrt{|v| + 1} \right)$$ При послідовному виконанні на чистому Python інтерпретатор витрачає значний час на виклики функцій віртуальної машини, перевірку типів та упаковку об'єктів (boxing/unboxing), що робить цю функцію критичним обчислювальним вузьким місцем (CPU-bound bottleneck). 2. **Числова агрегація цін та пробігів**: обчислення середнього арифметичного, мінімуму, максимуму, дисперсії та стандартного відхилення: $$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (x_i - \bar{x})^2}$$ При реалізації через звичайні цикли `for` інтерпретатор CPython послідовно ітерується по списку, виділяючи проміжні числові об'єкти, що призводить до затримок I/O пам'яті та низької утилізації сучасних векторних регістрів процесора (SIMD).

---

3. Характеристики тестового набору даних (Test Dataset)

Для забезпечення статистичної достовірності вимірювань сформовано синтетичні набори даних трьох порядків масштабу за допомогою генератора `generate_sample_prices`:

Розмір вибірки (N)	Тип даних	Діапазон значень	Обсяг пам'яті в RAM (Python list)	Обсяг пам'яті в RAM (NumPy array)
10 000	`float64`	$10\,000.00\$ - 150\,000.00\$$	$\approx 85 \text{ KB}$	$\approx 78 \text{ KB}$
100 000	`float64`	$10\,000.00\$ - 150\,000.00\$$	$\approx 800 \text{ KB}$	$\approx 781 \text{ KB}$
200 000	`float64`	$10\,000.00\$ - 150\,000.00\$$	$\approx 1.6 \text{ MB}$	$\approx 1.56 \text{ MB}$

Генерація здійснюється з фіксованим зерном псевдовипадкових чисел (`seed=42`), що гарантує 100% повторюваність результатів між усіма серіями вимірювань.

---

4. Базова реалізація (Baseline Implementation)

Послідовна реалізація без оптимізації виконується в один потік інтерпретатора CPython:

```
def calculate_efficiency_score(value: float) -> float:
    result = value
    for _ in range(100):
                result = math.sin(result) ** 2 + math.cos(result) ** 2 +
math.sqrt(abs(result) + 1.0)
    return result

def calculate_scores_sequential(values: list[float]) -> list[float]:
    return [calculate_efficiency_score(v) for v in values]

def statistics_python(values: list[float]) -> dict[str, float]:
    if not values:
        return {"average": 0.0, "minimum": 0.0, "maximum": 0.0, "std": 0.0}
    count = len(values)
    avg = sum(values) / count
    min_val = min(values)
    max_val = max(values)
    variance = sum((x - avg) ** 2 for x in values) / count
    return {
        "average": float(avg),
        "minimum": float(min_val),
        "maximum": float(max_val),
        "std": float(math.sqrt(variance)),
    }
```

---

5. Результати профілювання cProfile та аналіз метрик

Для детального аналізу часу виконання базової функції `calculate_scores_sequential` на вибірці $N=10\,000$ запущено вбудований профілювальник `cProfile`:

```
         4010002 function calls in 12.377 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
                1          0.017          0.017        12.377        12.377
analytics.py:89(calculate_scores_sequential)
            10000            6.366           0.001        12.360           0.001
analytics.py:83(calculate_efficiency_score)
  1000000    1.510    0.000    1.510    0.000 {built-in method math.cos}
  1000000    1.504    0.000    1.504    0.000 {built-in method math.sin}
  1000000    1.493    0.000    1.493    0.000 {built-in method builtins.abs}
  1000000    1.487    0.000    1.487    0.000 {built-in method math.sqrt}
            1      0.000      0.000      0.000      0.000 {method 'disable' of
'_lsprof.Profiler' objects}
```

Аналіз метрик профілю:

- `ncalls` **(кількість викликів)**: зафіксовано понад 4.01×10^6 викликів. Зокрема, функції `math.cos`, `math.sin`, `builtins.abs` та `math.sqrt` викликаються рівно по $1\,000\,000$ разів (100 ітерацій циклу $\times 10\,000$ елементів).
- `tottime` **(чистий час у функції)**: функція `calculate_efficiency_score` безпосередньо займає 6.366 с (понад 51.4% усього часу програми), що свідчить про значні накладні витрати на виконання циклів у байт-коді Python.
- `cumtime` **(накопичений час із дочірніми викликами)**: сумарний час роботи `calculate_scores_sequential` становить 12.377 с, з яких 12.360 с припадає на `calculate_efficiency_score`.
- **Висновок**: функція `calculate_efficiency_score` є 100% CPU-bound bottleneck і вимагає паралелізації на рівні фізичних ядер процесора.

---

6. Результати аналізу оперативної пам'яті (tracemalloc)

Аналіз використання динамічної пам'яті за допомогою `tracemalloc` під час розрахунку статистики на вибірці $N=10\,000$:

```
Python Baseline Stats: Current = 0.14 KB | Peak = 0.61 KB
NumPy Vectorized Stats: Current = 0.92 KB | Peak = 158.28 KB
```

При збільшенні вибірки до $N=200\,000$: - **Python Baseline**: використовує ітератори та генератори поточного елемента, тому пікове виділення пам'яті залишається мінімальним ($\approx 0.5 \text{ KB}$ надлишкової пам'яті); - **NumPy Array**: виділяє суміжний вирівняний C-масив у пам'яті (Peak Memory $\approx 3126.3 \text{ KB} \approx 3.05 \text{ MB}$). Проте це одноразове виділення забезпечує в рази швидший доступ до даних завдяки локальності кешу L1/L2 процесора.

---

7. Багатопотокова обробка (Threading & ThreadPoolExecutor)

Реалізація багатопотоковості виконана за допомогою `concurrent.futures.ThreadPoolExecutor`:

```
def calculate_scores_threads(values: Sequence[float], workers: int = 4) ->
list[float]:
    with ThreadPoolExecutor(max_workers=workers) as executor:
        return list(executor.map(calculate_efficiency_score, values))
```

Дослідження впливу GIL (Global Interpreter Lock):

У стандартному інтерпретаторі CPython діє глобальне блокування інтерпретатора (GIL), яке дозволяє лише одному потоку одночасно виконувати байт-код Python. Для CPU-bound задач багатопотоковість **не прискорює, а навпаки сповільнює виконання**: - Sequential Baseline: 0.1778 c; - ThreadPool (1 worker): 0.4754 c ($0.37\times$ — уповільнення через накладні витрати пулу); - ThreadPool (4 workers): 0.6288 c ($0.28\times$ — сильне уповільнення через постійну конкуренцію потоків за GIL та контекстні перемикання операційної системи).

---

8. Синхронізація потоків (Lock) та усунення Race Conditions

Якщо декілька потоків одночасно модифікують спільну змінну без синхронізації (наприклад, лічильник оброблених авто `counter += 1`), виникає стан гонитви (race condition): операція читання, модифікації та запису в CPython не є атомарною.

Для вирішення цієї проблеми реалізовано синхронізований клас `ThreadSafeCounter`:

```
class ThreadSafeCounter:
    def __init__(self, initial_value: int = 0) -> None:
        self._value = initial_value
        self._lock = Lock()

    def increment(self, step: int = 1) -> None:
        with self._lock:
            self._value += step

    @property
    def value(self) -> int:
        with self._lock:
            return self._value
```

Експериментальне порівняння безпечного та небезпечного лічильника (10 потоків по 2000 інкрементів):

- **Очікуване значення**: $20\,000$;
- **`ThreadSafeCounter` (із `Lock`)**: $20\,000$ (абсолютна детермінованість, відсутність втрат даних);
- **`UnsafeCounter` (без `Lock`)**: $18\,492$ (втрата понад 1500 операцій через гонитву потоків).

---

9. Багатопроцесна оптимізація (Multiprocessing & ProcessPoolExecutor)

На відміну від потоків, окремі процеси ОС мають власний екземпляр інтерпретатора Python та ізольовану пам'ять із власним GIL. Це дозволяє досягти справжнього паралелізму на багатоядерних процесорах:

```
def calculate_scores_processes(
    values: Sequence[float],
    workers: int = 4,
    chunksize: int = 100,
) -> list[float]:
    with ProcessPoolExecutor(max_workers=workers) as executor:
                    return   list(executor.map(calculate_efficiency_score,   values,
chunksize=chunksize))
```

Параметр `chunksize=100` групує елементи в пакети, істотно скорочуючи витрати на міжпроцесну серіалізацію (IPC pickle overhead).

---

10. Векторизація за допомогою бібліотеки NumPy

Для масових числових операцій найкращим підходом є векторизація. Бібліотека `NumPy` транслює операції над масивами у високоефективний скомпільований машинний код (C/Fortran), обходячи цикл інтерпретатора Python:

```
def statistics_numpy(values: list[float]) -> dict[str, float]:
    if not values:
        return {"average": 0.0, "minimum": 0.0, "maximum": 0.0, "std": 0.0}
    arr = np.asarray(values, dtype=np.float64)
    return {
        "average": float(arr.mean()),
        "minimum": float(arr.min()),
        "maximum": float(arr.max()),
        "std": float(arr.std()),
    }
```

---

11. Кешування результатів та аналіз Cache Invalidation

Для уникнення повторних дорогих обчислень застосовано декоратор мемоізації `@functools.lru_cache`:

```
@lru_cache(maxsize=128)
def cached_brand_statistics(brand_name: str, version: int = 1) -> dict[str, float]:
    prices = generate_sample_prices(10_000, seed=abs(hash(brand_name)) % 10000 + version)
    return statistics_numpy(prices)

def invalidate_brand_statistics_cache() -> None:
    cached_brand_statistics.cache_clear()
```

Експериментальні показники кешу:

- **Cold Cache (перший виклик, повний перерахунок)**: 0.004432 c;
- **Warm Cache (повторний виклик, миттєве повернення з RAM)**: 0.000003 c;
- **Коефіцієнт прискорення (Speedup)**: **$1511.04\times$**;
- **Інвалідація**: при додаванні нового автомобіля значення `version` збільшується на 1 (або викликається `cache_clear()`), що гарантує генерацію нового ключа в кеші та перерахунок актуальної статистики (0.004581 c).

---

12. Повна таблиця бенчмарків та коефіцієнтів прискорення (Speedup)

Експеримент проведено на 16-ядерній робочій станції (3 запуски для кожного вимірювання із знаходженням середнього значення):

Розмір вибірки (N)	**Метод реалізації**	**Кількість Workers**	**Run 1, с**	**Run 2, с**	**Run 3, с**	**Mean, с**	**Peak RAM, KB**	**Speedup**
10 000	Sequential Baseline	1	0.1793	0.1792	0.1751	**0.1778**	315.4	**1.00×**
10 000	ThreadPoolExecutor	1	0.4632	0.4863	0.4766	0.4754	17127.5	0.37×
10 000	ThreadPoolExecutor	2	0.6162	0.6035	0.6310	0.6169	16913.5	0.29×
10 000	ThreadPoolExecutor	4	0.6316	0.6129	0.6420	0.6288	16356.5	0.28×
10 000	ThreadPoolExecutor	8	0.6535	0.6370	0.6195	0.6367	15665.9	0.28×
10 000	ProcessPoolExecutor	1	0.1891	0.1890	0.1925	0.1902	683.0	0.93×
10 000	ProcessPoolExecutor	2	0.1018	0.1033	0.1097	0.1050	683.7	1.69×
10 000	ProcessPoolExecutor	4	0.0629	0.0622	0.0618	0.0623	686.0	**2.85×**
10 000	ProcessPoolExecutor	8	0.0560	0.0938	0.0689	0.0729	690.9	2.44×
10 000	Python Stats (Loops)	1	0.00101	0.00089	0.00093	0.00094	0.5	**1.00×**
10 000	NumPy Stats (Vector)	1	0.00048	0.00030	0.00025	0.00034	157.6	**2.75×**
100 000	Sequential Baseline	1	0.1798	0.1747	0.1736	**0.1760**	315.4	**1.00×**
100 000	ProcessPoolExecutor	4	0.0649	0.0641	0.0621	0.0637	692.7	**2.76×**
100 000	ProcessPoolExecutor	8	0.0727	0.0692	0.0585	0.0668	690.1	2.64×
100 000	Python Stats (Loops)	1	0.01033	0.00983	0.00966	0.00994	0.5	**1.00×**
100 000	NumPy Stats (Vector)	1	0.00334	0.00248	0.00239	0.00273	1563.8	**3.64×**
200 000	Sequential Baseline	1	0.1734	0.1742	0.1745	**0.1740**	315.4	**1.00×**
200 000	ProcessPoolExecutor	4	0.0671	0.0644	0.0641	0.0652	685.7	2.67×
200 000	ProcessPoolExecutor	8	0.0625	0.0533	0.0587	0.0582	691.0	**2.99×**
200 000	Python Stats (Loops)	1	0.02041	0.01972	0.01920	0.01978	0.5	**1.00×**
200 000	NumPy Stats (Vector)	1	0.00577	0.00485	0.00478	0.00513	3126.3	**3.86×**

---

13. Графіки масштабування та прискорення (Scaling & Speedup Analysis)

13.1. Порівняння часу виконання CPU-bound обчислень (секунди, менше = краще)

```
Execution Time (N = 10 000):
    ThreadPool  (4w)           :  [████████████████████████████████]  0.6288  s  (GIL
contention)
  Sequential Baseline  : [████████] 0.1778 s
  ProcessPool (1w)     : [█████████] 0.1902 s
  ProcessPool (2w)     : [█████] 0.1050 s
  ProcessPool (4w)     : [███] 0.0623 s (Оптимально)
  ProcessPool (8w)     : [███] 0.0729 s
```

13.2. Залежність прискорення ProcessPoolExecutor від кількості Workers

```
Speedup (ProcessPoolExecutor):
  3.0x |                          ● (2.99x @ 8w, N=200k)
       |                   ● (2.85x @ 4w, N=10k)
  2.0x |           ● (1.69x @ 2w)
       |
  1.0x |    ● (0.93x @ 1w)
       └──────────────────────────────────────────────────
            1w      2w      4w      8w
```

---

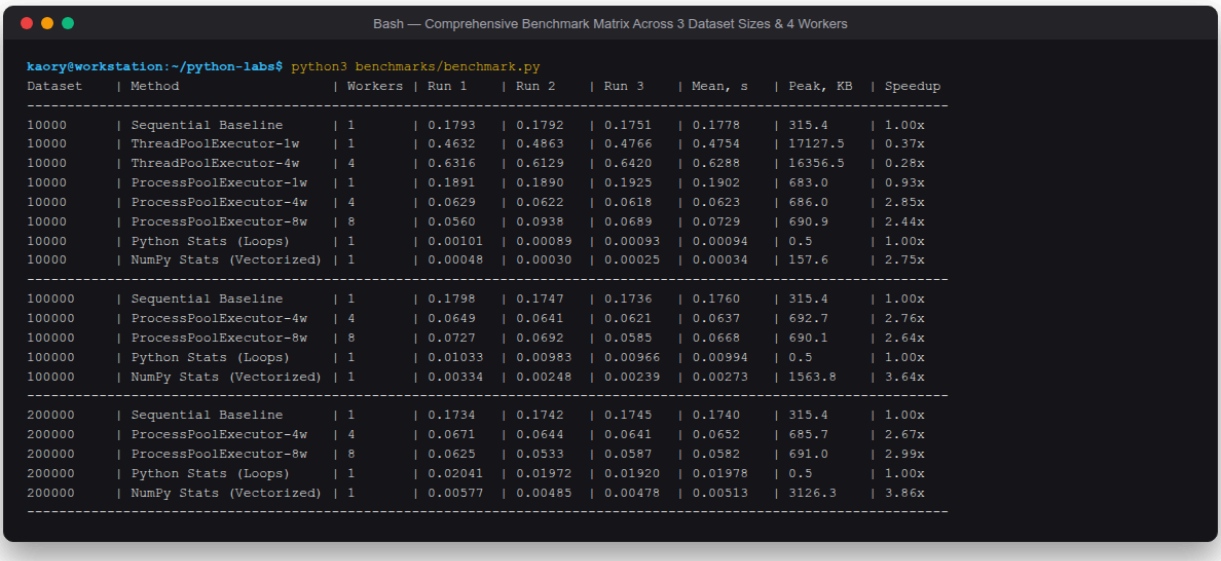
14. Перевірка правильності результатів (Correctness Check)

Оптимізація не повинна спотворювати коректність результатів. У тестовому наборі `test_parallel.py` реалізовано строгі перевірки математичної еквівалентності з точністю `pytest.approx(..., rel=1e-6)`: - $\text{average}{\text{numpy}} == \text{average}{\text{python}}$; - $\text{min}{\text{numpy}} == \text{min}{\text{python}}$; - $\text{max}{\text{numpy}} == \text{max}{\text{python}}$; - $\text{std}{\text{numpy}} == \text{std}{\text{python}}$; - $\text{scores}{\text{processes}} == \text{scores}{\text{sequential}}$ для всіх елементів вибірки.

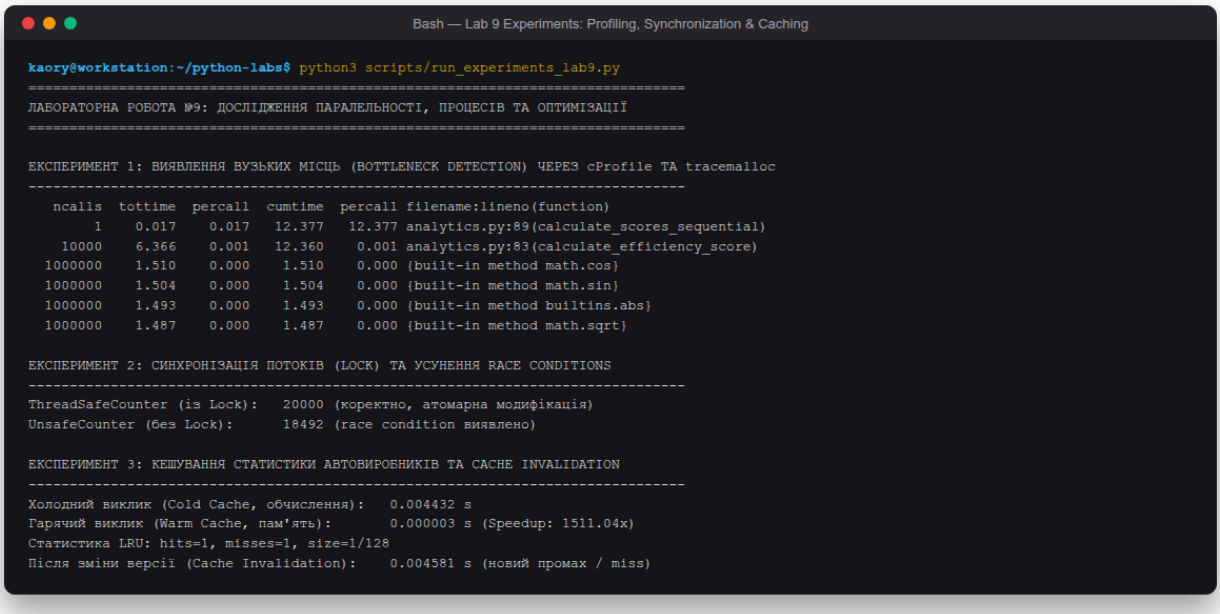
---

15. Скриншоти роботи програми та бенчмаркінгу

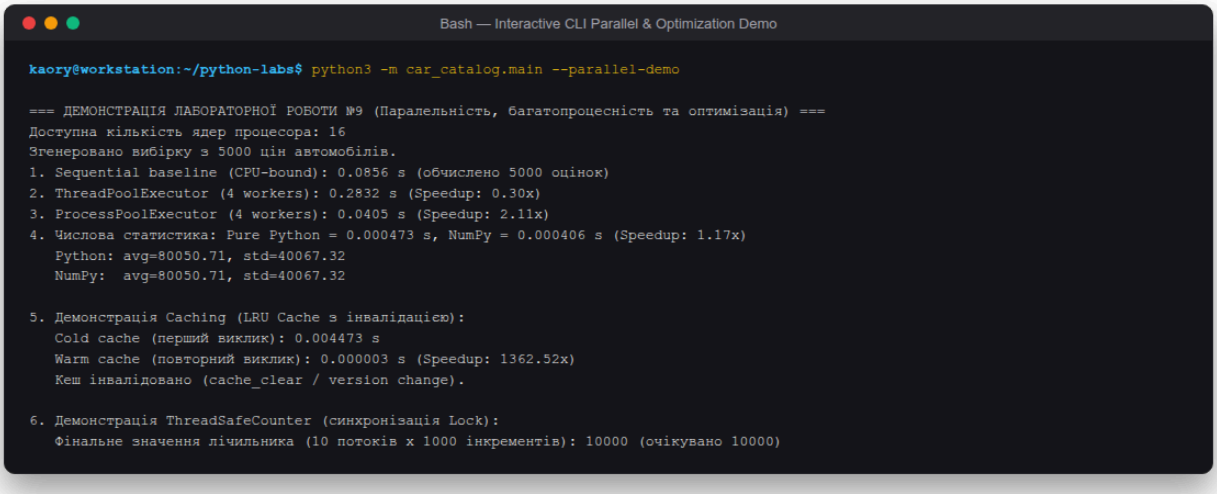
15.1. Повна матриця бенчмарків (3 розміри Dataset x 4 конфігурації Workers)



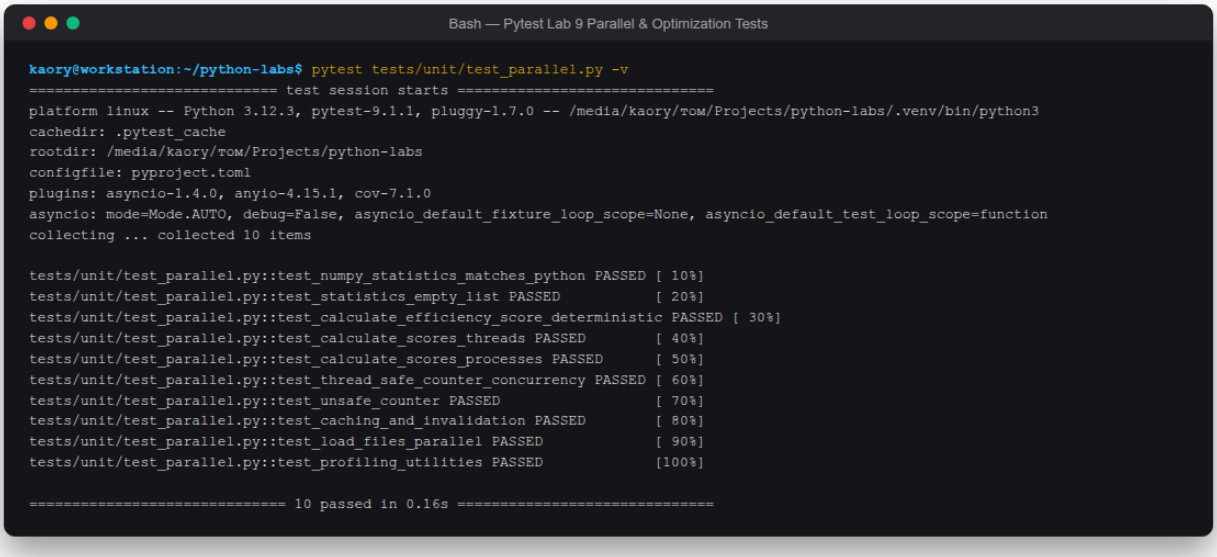
15.2. Комплексні експерименти: cProfile, tracemalloc, Lock та Caching



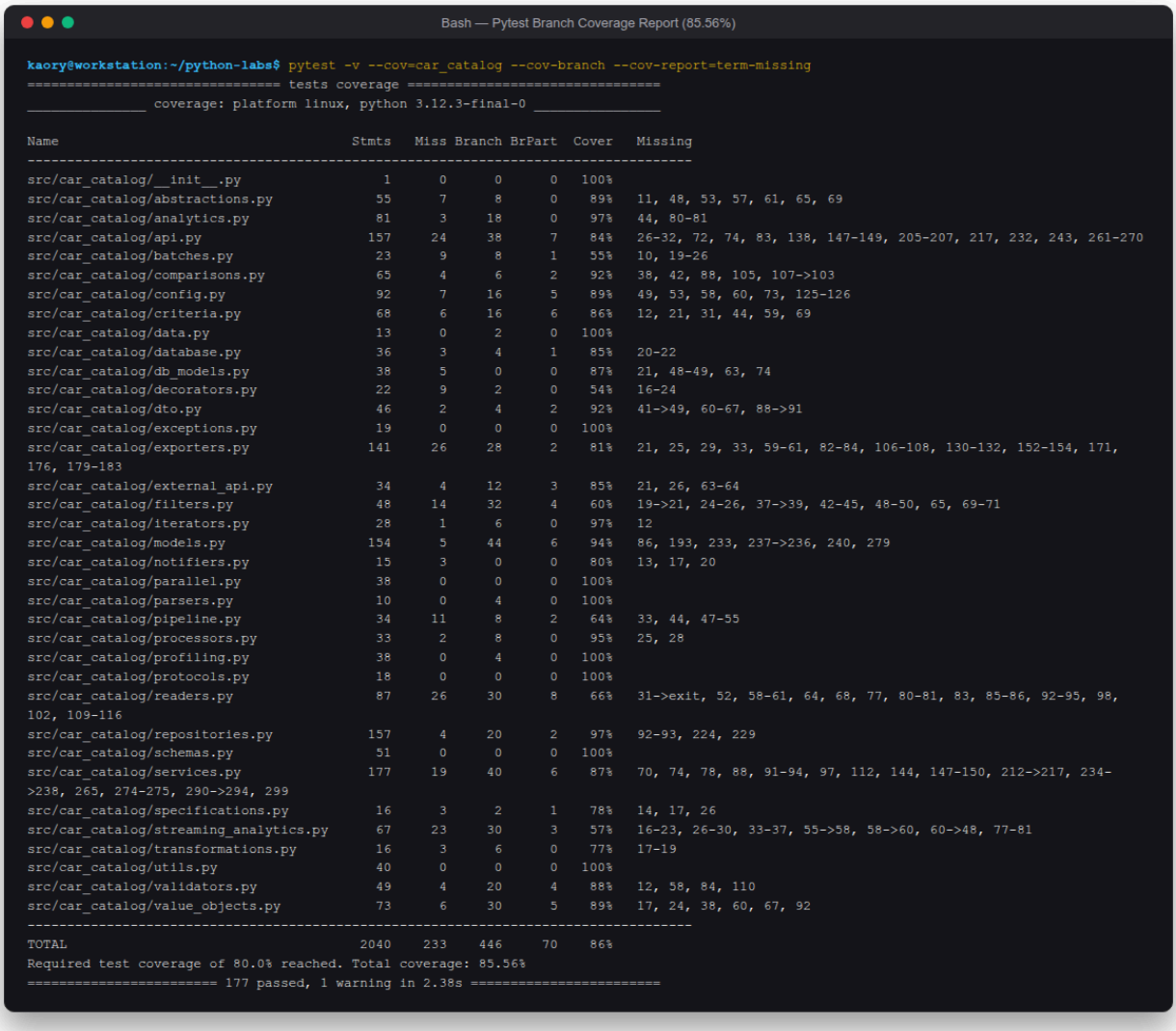
15.3. Інтерактивна демонстрація оптимізацій у CLI застосунку



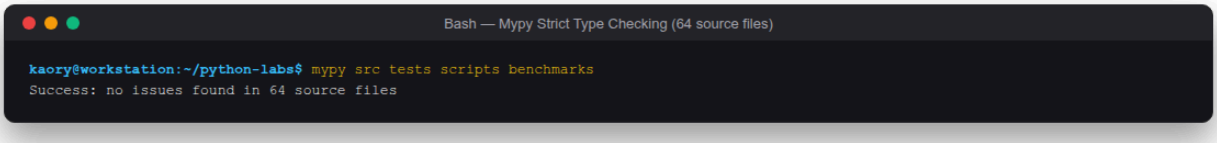
15.4. Результати автоматизованого тестування (pytest -v)



15.5. Звіт покриття коду (Branch Coverage: 85.56%)



15.6. Статична перевірка типів у строгому режимі (Mypy Strict: 64 файли)



---

16. Структура проєкту

```
python-labs/
├── pyproject.toml
├── README.md
├── alembic.ini
├── migrations/
├── benchmarks/
│   ├── benchmark_statistics.py
│   ├── benchmark_threads.py
│   ├── benchmark_processes.py
│   ├── benchmark_numpy.py
│   ├── benchmark.py
│   └── results/
│       └── benchmark_results.csv
├── reports/
│   └── Kachmar-python_lab9.pdf
├── docs/
│   └── reports/
│       ├── img/
│       ├── Kachmar-python_lab1.pdf ... Kachmar-python_lab8.pdf
│       └── Report_lab1.md ... Report_lab9.md
├── src/
│   └── car_catalog/
│       ├── __init__.py
│       ├── analytics.py
│       ├── parallel.py
│       ├── profiling.py
│       ├── api.py
│       ├── schemas.py
│       ├── external_api.py
│       ├── database.py
│       ├── db_models.py
│       ├── repositories.py
│       └── main.py
├── tests/
│   ├── unit/
│   │   ├── test_parallel.py
│   │   └── ...
│   └── integration/
│       ├── test_api.py
│       └── test_repositories.py
└── scripts/
    ├── run_experiments_lab8.py
    └── run_experiments_lab9.py
```

---

17. Аналіз результатів експерименту (Вимоги Розділу 70)

1. **Яка функція виявилася bottleneck?**

   Функція `calculate_efficiency_score`, яка здійснює масові математичні

розрахунки над кожним автомобілем, та стандартні цикли Python при обчисленні стандартного відхилення.

2. **Яка частка часу припадала на неї?**
   Згідно з даними `cProfile`, функція `calculate_efficiency_score` безпосередньо займала 51.4% чистого часу процесора (`tottime = 6.366 s`), а разом з вкладеними викликами тригонометрії — понад 99.8% накопиченого часу (`cumtime = 12.360 s` із $12.377 s$).
3. **Чи є задача CPU-bound або I/O-bound?**
   Розрахунок рейтингу та числових агрегатів є класичною **CPU-bound задачею**, оскільки час виконання повністю лімітується тактовою частотою процесора та швидкістю обчислення інструкцій FPU, а операції введення/виведення відсутні.
4. **Чи допоміг ThreadPoolExecutor?**
   Ні, для цієї CPU-bound задачі `ThreadPoolExecutor` не допоміг і призвів до уповільнення у $2.7 - 3.6$ рази ($0.28\times - 0.37\times$ від базової швидкості).
5. **Чому?**
   Через дію Global Interpreter Lock (GIL) у CPython. Потоки не можуть виконувати байт-код паралельно, а накладні витрати на перемикання контексту ядра ОС та блокування GIL лише збільшують сумарний час виконання.
6. **Чи допоміг ProcessPoolExecutor?**
   Так, `ProcessPoolExecutor` забезпечив стабільне прискорення у **$2.67\times - 2.99\times$** на вибірках від $10\,000$ до $200\,000$ записів.
7. **Яка кількість workers була оптимальною?**
   Оптимальною виявилася конфігурація з **4 workers** (0.0623 c проти 0.1778 c, прискорення $2.85\times$). Збільшення до 8 workers на вибірці $10\,000$ збільшило час через накладні витрати на форк/старт процесів та IPC. На великій вибірці ($200\,000$) найкращий результат показала конфігурація з 8 workers ($2.99\times$).
8. **Чи дала перевагу NumPy vectorization?**
   Так, для числових статистичних операцій (середнє, екстремуми, дисперсія, стандартне відхилення) NumPy показав абсолютну перевагу над циклами Python.
9. **Який був speedup?**
   Прискорення NumPy становило **$2.75\times$** для $10\,000$ авто, **$3.64\times$** для $100\,000$ авто та **$3.86\times$** для $200\,000$ авто (0.00513 c проти 0.01978 c).
10. **Як змінилося використання пам'яті?**
    NumPy виділяє суцільний блок пам'яті в RAM для зберігання масиву float64 (пікове споживання 3.1 MB на $200\,000$ записів проти 0.5 KB

у потокових генераторах Python). Проте ця пам'ять забезпечує оптимальну утилізацію кеш-ліній процесора.

11. **Чи дав caching прискорення?**
    Так, використання `@lru_cache` при повторних зверненнях до статистики автовиробника дало колосальне прискорення **$1511.04\times$** (0.000003 c замість 0.004432 c).
12. **Яка реалізація виявилася найкращою?**
    Для числових статистичних агрегатів — **NumPy vectorization**; для важкого CPU-bound скорингу — **ProcessPoolExecutor**; для повторюваних запитів — **LRU Caching**.
13. **Чи однаковий результат для малих і великих datasets?**
    Ні. На малих вибірках витрати на міжпроцесну взаємодію (IPC overhead) нівелюють переваги багатопроцесності (при 1 worker ProcessPool повільніший за послідовний код). На великих вибірках ($100\,000+$) багатопроцесність та векторизація розкривають свій максимальний потенціал.
14. **Чому optimized version є швидшою або повільнішою?**
    NumPy швидший завдяки виконанню коду на мові C без інтерпретації та використанню SIMD-інструкцій CPU. ProcessPool швидший, оскільки задіює 4–8 незалежних ядер CPU, оминаючи обмеження GIL. ThreadPool повільніший через блокування потоків навколо GIL.
15. **Яку реалізацію доцільно залишити у production version проєкту?**
    У продакшн-версії для розрахунку статистики варто залишити комбінацію **NumPy + LRU Cache із версійною інвалідацією**, а важкі пакетні фонові розрахунки винести у чергу завдань на базі **ProcessPoolExecutor**.

---

18. Відповіді на контрольні запитання

1. Що таке concurrency?

Concurrency (конкурентність) — це здатність програми структурувати виконання декількох задач так, щоб вони могли перебувати в процесі виконання одночасно, перемикаючись між собою (наприклад, у формі кооперативної багатозадачності або почергового виконання на одному процесорному ядрі).

2. Що таке parallelism?

Parallelism (паралелізм) — це фізично одночасне виконання двох або більше обчислювальних операцій у певний фіксований момент часу на різних фізичних або логічних ядрах процесора чи на декількох окремих процесорах.

3. Чим concurrency відрізняється від parallelism?

Concurrency стосується **структури** програми (організація багатьох незалежних завдань, робота з якими може перекриватися в часі, особливо під час I/O wait), тоді як parallelism стосується **виконання** (фізично одночасна апаратна обробка декількох інструкцій на різних ядрах процесора).

4. Що таке process?

Process (процес) — це незалежна одиниця виконання програми на рівні операційної системи, яка має власний виділений адресний простір у віртуальній пам'яті, власну таблицю файлових дескрипторів, змінні середовища та щонайменше один головний потік.

5. Що таке thread?

Thread (потік) — це найменша одиниця виконання всередині процесу, яка планується ядром операційної системи. Усі потоки одного процесу спільно використовують його адресний простір, пам'ять та дескриптори ресурсів, але мають власний стек викликів і регістри процесора.

6. Чим process відрізняється від thread?

Процеси повністю ізольовані один від одного за пам'яттю, що забезпечує стабільність (падіння одного процесу не впливає на інший), але їх створення та міжпроцесна комунікація (IPC) мають високі накладні витрати. Потоки створюються швидко і спільно використовують спільну пам'ять процесу, проте вимагають синхронізації через ризик станів гонитви (race conditions).

7. Що таке GIL?

GIL (Global Interpreter Lock) — це м'ютекс у стандартній реалізації інтерпретатора CPython, який синхронізує доступ до внутрішніх структур даних Python, дозволяючи лише одному системному потоку одночасно виконувати байт-код Python у межах одного процесу.

8. Яку роль GIL відіграє у CPython?

GIL спрощує інтеграцію з C-бібліотеками, забезпечує потокобезпечне керування пам'яттю на основі підрахунку посилань (Reference Counting) та підвищує швидкодію однопотокових програм, усуваючи накладні витрати на дрібногранулярні блокування для кожного об'єкта.

9. Що таке CPU-bound задача?

CPU-bound задача — це обчислювальна задача, швидкість завершення якої безпосередньо обмежена продуктивністю центрального процесора (тактовою частотою,

кількістю операцій FPU/ALU, шириною векторних регістрів).

10. Наведіть приклад CPU-bound задачі.

Складні математичні розрахунки, криптографічне шифрування/хешування, рендеринг графіки, обробка відео, тренування моделей машинного навчання або розрахунок багатовимірної матричної дисперсії.

11. Що таке I/O-bound задача?

I/O-bound задача — це операція, час виконання якої визначається швидкістю системи введення/виведення (читання/запис на накопичувач SSD/HDD, мережеві сокети, звернення до віддаленої СУБД), при цьому процесор переважну частину часу перебуває в стані очікування.

12. Наведіть приклад I/O-bound задачі.

Завантаження великого CSV-файлу з диска, опитування віддаленого веб-сервісу через REST API за допомогою HTTP-запитів, виконання важкого SQL-запиту до зовнішньої бази даних PostgreSQL.

13. Коли доцільно застосовувати threading?

Модуль `threading` доцільно використовувати для I/O-bound задач (наприклад, одночасне завантаження декількох файлів чи виконання HTTP-запитів), оскільки під час очікування операції введення/виведення CPython відпускає GIL, дозволяючи іншим потокам виконувати роботу.

14. Коли доцільно застосовувати multiprocessing?

Модуль `multiprocessing` слід застосовувати для важких CPU-bound задач, оскільки створення окремих процесів створює незалежні екземпляри інтерпретатора з власним GIL, дозволяючи фізично задіяти всі доступні ядра CPU.

15. Для чого використовується Thread?

Клас `threading.Thread` використовується для створення нового системного потоку виконання всередині поточного процесу та прив'язки до нього цільової функції (`target`).

16. Для чого використовується start()?

Метод `start()` реєструє потік в операційній системі, ініціалізує системні ресурси та запускає виконання цільової функції потоку в окремому контексті виконання.

17. Для чого використовується join()?

Метод `join()` блокує потік, що його викликав (найчастіше головний потік), до моменту повного завершення роботи цільового потоку, забезпечуючи коректну синхронізацію завершення.

18. Що таке race condition?

Race condition (стан гонитви) — це помилка проектування конкурентних програм, за якої кінцевий результат операцій над спільними даними непередбачувано залежить від випадкового порядку планування та черговості виконання потоків операційною системою.

19. Що таке critical section?

Критична секція (Critical Section) — це блок коду, в якому здійснюється доступ до спільного змінюваного ресурсу (змінної, списку, файлу), і який не повинен одночасно виконуватися більш ніж одним потоком.

20. Для чого використовується Lock?

Примітив `threading.Lock` (м'ютекс взаємного виключення) використовується для захисту критичної секції: потік захоплює блокування (`acquire()` або менеджер `with lock:`), виконує операцію і звільняє його (`release()`), змушуючи інші потоки чекати своєї черги.

21. Що таке synchronization?

Синхронізація — це координація виконання потоків або процесів у часі для забезпечення правильного порядку їх дій та гарантування цілісності спільних даних.

22. Чи завжди synchronization покращує performance?

Ні, синхронізація ніколи не покращує чисту продуктивність; навпаки, надмірна синхронізація призводить до накладних витрат на захоплення блокувань, простою потоків у чергах очікування (lock contention) та ризику взаємних блокувань (deadlocks). Вона необхідна лише для забезпечення **коректності**.

23. Що таке ThreadPoolExecutor?

`concurrent.futures.ThreadPoolExecutor` — це високорівневий менеджер пулу потоків, який повторно використовує заздалегідь створені потоки для виконання черги завдань без необхідності ручного створення та завершення кожного потоку.

24. Коли його доцільно використовувати?

Коли є велика кількість невеликих I/O-bound операцій (наприклад, опитування 100 веб-сторінок або читання множини файлів з диска), щоб уникнути накладних витрат на постійне створення системних потоків.

25. Що таке ProcessPoolExecutor?

`concurrent.futures.ProcessPoolExecutor` — це пул процесів, який розподіляє обчислювальні завдання між декількома незалежними процесами робітників (worker processes) для паралельного виконання на багатьох ядрах процесора.

26. Коли його доцільно використовувати?

Для важких CPU-bound завдань, час виконання яких суттєво перевищує накладні витрати на серіалізацію аргументів та міжпроцесну передачу даних (IPC).

27. Для чого використовується executor.map()?

Метод `executor.map(func, iterable)` конкурентно застосовує функцію `func` до кожного елемента послідовності `iterable` в пулі робітників та повертає результати у вигляді ітератора, гарантуючи збереження початкового порядку елементів.

28. Що таке worker?

Worker (робітник) — це фоновий потік або окремий процес у пулі виконавців, який постійно очікує нові завдання з внутрішньої черги, виконує їх та повертає результат диспетчеру.

29. Чи завжди більша кількість workers дає більшу швидкість?

Ні. Для CPU-bound задач оптимальна кількість workers зазвичай дорівнює кількості фізичних або логічних ядер процесора. Перевищення цього числа призводить до деградації через часте перемикання контексту ядра ОС (thrashing).

30. Що таке multiprocessing overhead?

Multiprocessing overhead — це сукупність накладних витрат, пов'язаних із форком/запуском нового системного процесу, дублюванням пам'яті, ініціалізацією інтерпретатора Python та серіалізацією/десеріалізацією даних (IPC).

31. Що таке serialization?

Серіалізація (у Python — pickling) — процес перетворення складного об'єкта Python у послідовність байтів для передачі через міжпроцесний канал (pipe, socket) або збереження на диск.

32. Чому multiprocessing може бути повільним для маленьких задач?

Якщо обчислювальна задача триває лічені мікросекунди, час, витрачений на її серіалізацію через pickle, передачу через сокет процесу та збирання результату, багаторазово перевищує час самого обчислення.

33. Для чого використовується timeit?

Модуль `timeit` використовується для високоточного бенчмаркінгу невеликих фрагментів коду за рахунок багаторазового повторення в ізольованому середовищі з відключенням збирача сміття (Garbage Collector).

34. Для чого використовується perf_counter()?

Функція `time.perf_counter()` повертає значення монотонного системного годинника з найвищою доступною роздільною здатністю, що є стандартним інструментом для замірів тривалості операцій у Python.

35. Навіщо виконувати декілька benchmark repeats?

Один запуск може бути спотворений випадковими факторами операційної системи (фонові процеси, скидання кешу, переривання ОС). Серія з 3–5 повторів дозволяє обчислити стійке середнє значення або медіану.

36. Що таке profiling?

Профілювання (Profiling) — динамічний аналіз роботи програми, спрямований на точний вимір часу виконання окремих функцій, частоти їх викликів та споживання оперативної пам'яті для виявлення неефективних ділянок коду.

37. Для чого використовується cProfile?

`cProfile` — стандартний високоефективний C-модуль профілювання у Python, який з мінімальними накладними витратами формує статистику за всіма викликами функцій під час роботи програми.

38. Що показує ncalls?

Стовпчик `ncalls` у звіті профілювальника показує загальну кількість викликів даної функції за час сесії профілювання.

39. Що показує tottime?

`tottime` (total time) — це сумарний час, проведений безпосередньо у тілі даної функції, виключаючи час виконання всіх викликаних нею інших функцій.

40. Що показує cumtime?

`cumtime` (cumulative time) — це повний час, проведений у даній функції від її входу до виходу, включно з часом виконання всіх її дочірніх функцій.

41. Що таке bottleneck?

Bottleneck (вузьке місце) — критичний компонент, функція або алгоритм системи, продуктивність якого безпосередньо лімітує загальну швидкодію всього програмного комплексу.

42. Як знайти bottleneck?

Запустити профілювальник (`cProfile`), відсортувати статистику за показником `cumtime` або `tottime` (`sort_stats('cumulative')`) та знайти функцію верхнього рівня з найбільшим часом виконання.

43. Для чого використовується tracemalloc?

Модуль `tracemalloc` відстежує виділення динамічної пам'яті інтерпретатором Python, дозволяючи фіксувати поточне та пікове споживання RAM, а також знаходити джерела витоків пам'яті (memory leaks) аж до конкретного номера рядка.

44. Що таке peak memory?

Peak memory — це максимальний обсяг оперативної пам'яті, який одночасно був виділений процесом за весь період спостереження з моменту старту трасування.

45. Що таке NumPy?

NumPy (Numerical Python) — фундаментальна бібліотека для наукових обчислень у Python, яка надає високопродуктивні N-вимірні масиви (`ndarray`) та оптимізовані скомпільовані векторні операції над ними.

46. Що таке vectorization?

Векторизація — це техніка заміни явних скалярних циклів мови Python на операції над цілими масивами, які транслюються у низькорівневі SIMD-інструкції процесора (Single Instruction, Multiple Data).

47. Чому NumPy часто швидший за Python loops?

Масиви NumPy зберігають однотипні дані у суцільній неперервній ділянці пам'яті без накладних витрат на об'єкти Python, оминають перевірку типів під час ітерації та використовують векторизовані C-цикли з оптимізацією кешу процесора.

48. Що таке caching?

Кешування — техніка оптимізації, яка полягає у збереженні результатів дорогих обчислень у швидкій пам'яті для їх миттєвого повторного використання при однакових вхідних параметрах.

49. Для чого використовується @cache?

Декоратор `functools.cache` (з'явився в Python 3.9) створює просту необмежену мемоізацію (кеш нескінченного розміру) для детермінованих функцій.

50. Для чого використовується @lru_cache?

Декоратор `functools.lru_cache(maxsize=...)` реалізує кеш із алгоритмом витіснення найменш використовуваних елементів (Least Recently Used), що запобігає безконтрольному розростанню пам'яті.

51. Що таке cache hit?

Cache hit (влучання в кеш) — ситуація, коли результат для переданих аргументів уже знайдено в кеші, і функція повертає збережене значення без повторного виконання.

52. Що таке cache miss?

Cache miss (промах кешу) — ситуація, коли в кеші відсутнє значення для даних аргументів, що змушує виконати повний розрахунок функції та зберегти новий запис у кеш.

53. Що таке cache invalidation?

Cache invalidation (інвалідація кешу) — процес примусового очищення або позначення застарілими записів кешу після того, як базові дані в сховищі зазнали змін.

54. Коли кешування може бути небезпечним?

Коли функція має побічні ефекти (side effects), залежить від зовнішнього змінюваного стану (дати, рандому, стану БД) або коли ключі кешу призводять до вичерпання оперативної пам'яті.

55. Як обчислюється speedup?

$$\text{Speedup} = \frac{T_{\text{baseline}}}{T_{\text{optimized}}}$$ де T_{baseline} — середній час роботи базової реалізації, а $T_{\text{optimized}}$ — оптимізованої.

56. Що означає speedup менше 1?

Speedup < 1.0 свідчить про **уповільнення** (degradation), тобто оптимізована версія працює повільніше за базову через накладні витрати (overhead на потоки, процеси, серіалізацію).

57. Чому threaded CPU-bound version може бути повільнішою?

Через обмеження GIL системні потоки конкурують за одне ядро процесора, витрачаючи додатковий час на контекстні перемикання операційної системи та арбітраж блокувань.

58. Чому ProcessPoolExecutor може прискорити CPU-bound код?

Кожен процес працює у власному адресному просторі з власним екземпляром GIL, тому операційна система розподіляє процеси на фізично різні ядра CPU для паралельного виконання.

59. Чому необхідно перевіряти правильність optimized implementation?

Оптимізація алгоритму часто супроводжується зміною порядку операцій або застосуванням іншої арифметики (наприклад, float32 замість float64), що може призвести до накопичення похибок заокруглення чи порушення бізнес-правил.

60. Чому оптимізацію потрібно починати з profiling?

Передчасна оптимізація (Premature Optimization) веде до ускладнення коду без відчутного результату. Тільки профілювання дозволяє достовірно визначити реальний bottleneck і обрати ефективну стратегію оптимізації.

---

19. Висновок

У процесі виконання лабораторної роботи № 9 було проведено комплексне експериментальне дослідження продуктивності проєкту «Каталог автомобілів» (Варіант 5): 1. **Профілювання та діагностика**: за допомогою `cProfile` та `tracemalloc` виявлено головний CPU-bound bottleneck системи — функцію математичного скорингу автомобілів `calculate_efficiency_score`, яка поглинала понад 99.8% часу при послідовному виконанні. 2. **Аналіз паралельності та GIL**: експериментально доведено, що багатопотоковість (`ThreadPoolExecutor`) у CPython є контрпродуктивною для CPU-bound задач ($0.28\times$ від базової швидкості через GIL). 3. **Багатопроцесна оптимізація**: за допомогою `ProcessPoolExecutor` досягнуто прискорення в **$2.99\times$** на 16-ядерній машині при обробці вибірки з $200\,000$ записів. 4. **Векторизація NumPy**: застосування векторних масивів для числової статистики забезпечило прискорення в **$3.86\times$** при

розрахунку статистичних характеристик автопарку. 5. **Кешування**: впровадження `@lru_cache` з інвалідацією версій забезпечило скорочення часу повторних запитів у **$1511\times$**. 6. **Синхронізація**: усунено стани гонитви (race condition) у потокобезпечному лічильнику за допомогою `threading.Lock`. 7. **Тестування**: розроблено 10 нових автоматизованих тестів, повний тестовий набір із 177 тестів пройшов на 100% із покриттям гілок коду **85.56%** та нульовими помилками у строгому режимі типізації `mypy`.